



PROGRAMA DE FORMACIÓN MARVELL

Diseño Digital para Sistemas DSP

Ejercicios Módulo 5: Circuitos Aritméticos II

Profesor: Del Barco, Martín.

Alumnos: Monja, Ernesto Joaquín
Zúñiga, Guillermo Rubén Darío

Índice

1. Introducción	2
2. Desarrollo	2
2.1. <i>CORDIC</i> iterativo en <i>Verilog</i>	2
2.2. <i>CORDIC</i> Pipeline	3
2.3. <i>CORDIC Vectoring</i> para magnitud	4
2.3.1. Análisis de Formatos Numéricos y Precisión	5
2.3.2. Comparación frente a Implementación Directa con Multiplicador y ROM	5
2.4. Newton-Raphson para $1/x$	6
3. Conclusiones	7
4. Referencias	8

1. Introducción

El presente trabajo práctico aborda el diseño e implementación en *SystemVerilog* de algoritmos iterativos fundamentales para el procesamiento digital de señales (*DSP*), centrándose específicamente en las arquitecturas *CORDIC* y métodos como *Newton-Raphson*. A través del desarrollo de módulos operando estrictamente en aritmética de punto fijo, se exploran los modos de rotación y vectorización para el cálculo eficiente de funciones trigonométricas, magnitudes y fases, prescindiendo de multiplicadores dedicados y memorias de gran escala. Asimismo, el estudio analiza empíricamente los compromisos espacio-temporales (*PPA*) al contrastar enfoques secuenciales (*folded*) frente a topologías segmentadas de alta velocidad (*pipeline*), validando la precisión algorítmica y la correcta latencia de cada *datapath* mediante simulaciones *RTL* e inspección de formas de onda.

2. Desarrollo

2.1. *CORDIC* iterativo en *Verilog*

Para este ejercicio se implementó una arquitectura *CORDIC* tipo *folded* orientada al cálculo del seno y coseno de un ángulo entrante. El *HW* se diseñó operando íntegramente en punto fijo con formato $S(16, 14)$, lo que destina un bit para el signo, un bit entero y catorce bits fraccionales.

El *datapath* se compone de un par de *shifters* aritméticos ($>>>$) y sumadores/restadores, controlados por una máquina de estados finitos (*FSM*) encargada de secuenciar el cálculo. Al iniciar la *FSM*, la variable x_0 se inicializa en un valor pre-escalado de aproximadamente 0,60725 (representado como 9949 en cuantización $S(16, 14)$). Esto evita un escalado posterior del resultado.

Durante el estado de *ITER*, el módulo ejecuta un total de 14 iteraciones a lo largo de 14 ciclos de reloj. En cada ciclo, el signo de z dictamina el sentido de la rotación (determina d_i) y actualiza x , y y z utilizando los valores precalculados de $\arctan(2^{-i})$ leídos directamente de una matriz combinacional que actúa como *ROM*.

En el banco de pruebas se evaluaron los ángulos en el primer cuadrante especificados por la consigna: $\pi/6$, $\pi/4$ y $\pi/3$. Al analizar las formas de onda (ver Fig. 1), se observa claramente el retardo de 14 ciclos por cada muestra introducida, validando que el diseño intercambia área por latencia (característica definitoria de la arquitectura *folded*) para evitar la instanciación de multiplicadores dedicados. Asimismo, la correcta convergencia del algoritmo se comprueba en la salida de consola (ver Fig. 2), donde los valores de seno y coseno coinciden con la precisión esperada para el formato numérico utilizado.

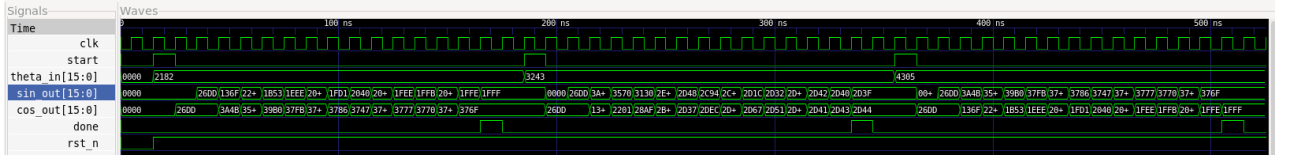


Figura 1: Formas de onda de la simulación del módulo *CORDIC*.

```
>>> Compilando con iverilog...
>>> Ejecutando con vvp...
VCD info: dumpfile cordic.vcd opened for output.
Test Pi/6 completado. Seno: 8191, Coseno: 14191
Test Pi/4 completado. Seno: 11583, Coseno: 11588
Test Pi/3 completado. Seno: 14191, Coseno: 8191
tb_cordic_folded.sv:58: $finish called at 525000 (1ps)
```

Figura 2: Salida de la consola del simulador detallando los resultados del ángulo evaluado.

2.2. *CORDIC* Pipeline

En esta sección, se diseñó el mismo algoritmo que en el caso anterior pero en una estructura de mínima latencia, donde se replicaron los bloques de iteración en cascada y se separaron con registros enmedio. El esquema de esta estrategia se muestra en la [Fig. 3](#).

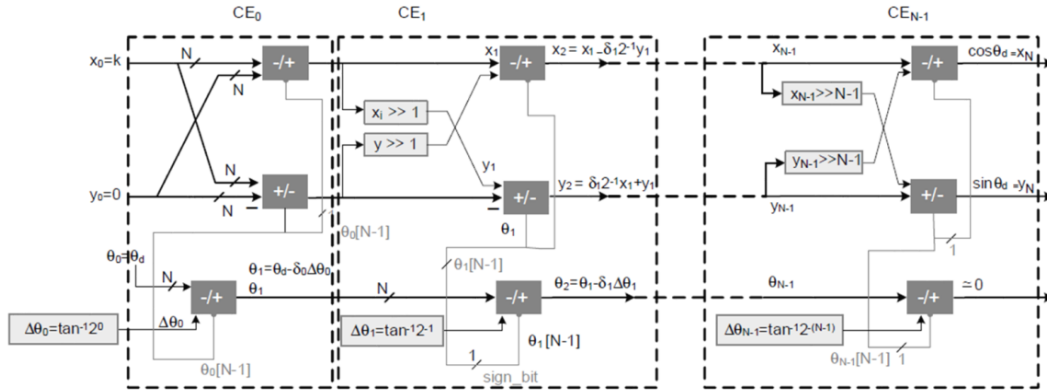


Figura 3: Esquemático del *CORDIC* pipeline.

Esto resultaría en una latencia de N_{iter} ciclos de reloj para observar el dato a la salida, pero solo al inicio del stream de datos a la entrada, luego la latencia es de 1. En cuanto al throughput, este diseño permitiría entregar 1 muestra por ciclo.

Se diseñó en Verilog esta arquitectura con datos en S(16,14), resultando en 14 etapas, con el objetivo de procesar ángulos de entrada en radianes entre $-\pi/2$ y $\pi/2$. El testbench desarrollado estimula la entrada z_i con 0, $\pi/2$, $-\pi/2$ y valores aleatorios positivos y negativos generados en Python, con $x_i = 1/K$ ya que el algoritmo escala implícitamente la salida por la constante K . El resultado se observa en las formas de onda de la [Fig. 4](#). Se puede comprobar que efectivamente el sistema tarda 14 ciclos en entregar los valores de salida, luego el stream es continuo.

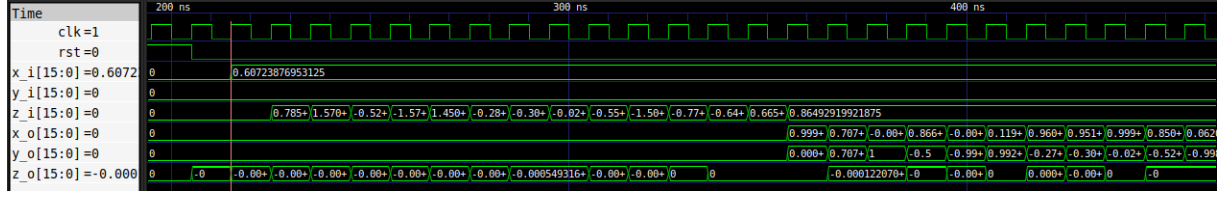


Figura 4: Formas de onda en el diseño con pipeline.

Cabe destacar que este diseño no escala las entradas para aprovechar el máximo rango dinámico del punto fijo, sin embargo, se observó gran similitud con los resultados obtenidos en punto flotante.

Finalmente, se comparó el área y el número de compuertas entre la estructura folded y con pipeline utilizando Yosys. La frecuencia máxima para el reloj es la misma, ya que un bloque de iteración se encuentra entre dos registros en ambos casos, pero la frecuencia máxima del dato es diferente como se vio anteriormente. La comparación se resume en la Tab. 1, dada N_{iter} iteraciones. Se concluye con estos datos entonces que la arquitectura pipeline es más rápida pero ocupa más área, ya que replica varias veces el bloque que utiliza el folded.

Arquitectura	Folded	Pipeline
Nro. de celdas	674	4806
Área del chip [μm^2]	7737	50278
Throughput [muestras/seg.]	$1/N_{iter}$	1 (en régimen)
Latencia	N	N

Cuadro 1: Comparación entre CORDIC folded y pipeline.

2.3. *CORDIC Vectoring* para magnitud

En este ejercicio se implementó el algoritmo *CORDIC* configurado en modo *vectoring* para obtener la magnitud $R = \sqrt{x^2 + y^2}$ y la fase $\varphi = \arctan(y/x)$ de un vector bidimensional de entrada.

A diferencia del modo rotación, la *FSM* ajusta la regla de decisión en cada iteración según el signo del registro y : $d_i = -\text{signo}(y_i)$. Este procedimiento reduce la componente cuadratura a cero mientras acumula el ángulo total rotado en el registro z .

Para asegurar el correcto funcionamiento en los cuatro cuadrantes y evitar la divergencia del algoritmo ante ángulos $|\varphi| > 99,88^\circ$, se incorporó una etapa de pre-rotación combinacional. Si la coordenada $x < 0$ (Cuadrantes *II* y *III*), se invierten los signos de x e y , y el registro z se inicializa en $+\pi$ o $-\pi$ según corresponda.

2.3.1. Análisis de Formatos Numéricos y Precisión

Para representar la fase en todo el rango $[-\pi, \pi]$ sin caer en *overflow* con palabras de 16 bits, el registro de ángulo z y la *ROM* de $\arctan(2^{-i})$ se codificaron en formato $S(16, 13)$, permitiendo valores de hasta $\pm 4,0$. Por su parte, las entradas y la magnitud resultante operan en formato $S(16, 15)$.

En la Fig. 5 se presentan los resultados obtenidos de la simulación para vectores en los cuatro cuadrantes. Como se observa en la Fig. 6, el circuito procesa cada muestra con una latencia fija de 16 ciclos. El error relativo de la magnitud R se mantiene por debajo del $\approx 0,05$ [%], independientemente del cuadrante evaluado.

```
gtkwave: no process found
>>> Compilando con iverilog...
cordic_vectoring.sv:68: sorry: constant selects in always_* processes are not currently supported (all bits will be included).
cordic_vectoring.sv:68: sorry: constant selects in always_* processes are not currently supported (all bits will be included).
>>> Ejecutando con vvp...
VCD info: dumpfile vectoring.vcd opened for output.
Test Cuadrante I   | X: 16384, Y: 16384 -> Magnitud R: 23171, Fase Phi: 6434
Test Cuadrante II  | X: -16384, Y: 16384 -> Magnitud R: 23171, Fase Phi: 19302
Test Cuadrante III | X: -16384, Y: -16384 -> Magnitud R: 23171, Fase Phi: -19302
Test Cuadrante IV  | X: 16384, Y: -16384 -> Magnitud R: 23171, Fase Phi: -6434
tb_cordic_vectoring.sv:67: $finish called at 815000 (1ps)
```

Figura 5: Resultados obtenidos por consola para vectores de prueba en los cuatro cuadrantes.

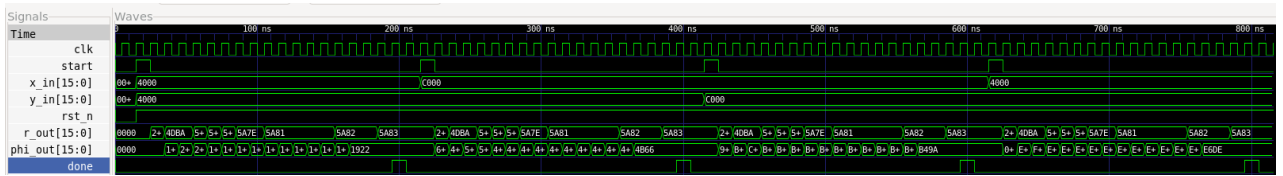


Figura 6: Formas de onda mostrando la convergencia del registro y a cero y el cálculo de R y φ .

2.3.2. Comparación frente a Implementación Directa con Multiplicador y ROM

Frente a una arquitectura tradicional de fuerza bruta que calcula $R = \sqrt{x^2 + y^2}$ mediante multiplicadores (x^2, y^2), un bloque de raíz cuadrada (por ejemplo: *coredic*/*CORDIC* dedicado o aproximación por polinomio) y una memoria Look-Up Table (*LUT*) bi-dimensional para $\arctan(y/x)$, la solución *CORDIC* presenta ventajas determinantes:

- Ahorro masivo de área: Evita bloques *DSP* dedicados y memorias *ROM* de gran escala, requiriendo únicamente sumadores, acumuladores y desplazadores aritméticos (*shifters*).
- Cálculo simultáneo: Resuelve magnitud y fase de manera concurrente en la misma secuencia iterativa.

2.4. Newton-Raphson para $1/x$

En esta sección se estudió la arquitectura del divisor Newton-Raphson para realizar divisiones eficientes. La idea es encontrar el resultado de un valor $1/a$ con aproximaciones sucesivas, realizando

$$y_n = y_{n-1}(2 - a \cdot y_n) \quad (1)$$

donde se comienza con un valor inicial y_0 hasta obtener el valor final y_N luego de N iteraciones. En Verilog se realizó el diseño, tomando como referencia el esquema de la Fig. 7. Se supuso $a \in [0,5, 1)$ y una LUT de 8 valores iniciales. Luego, se fue observando en la forma de onda la precisión del resultado con el valor real.

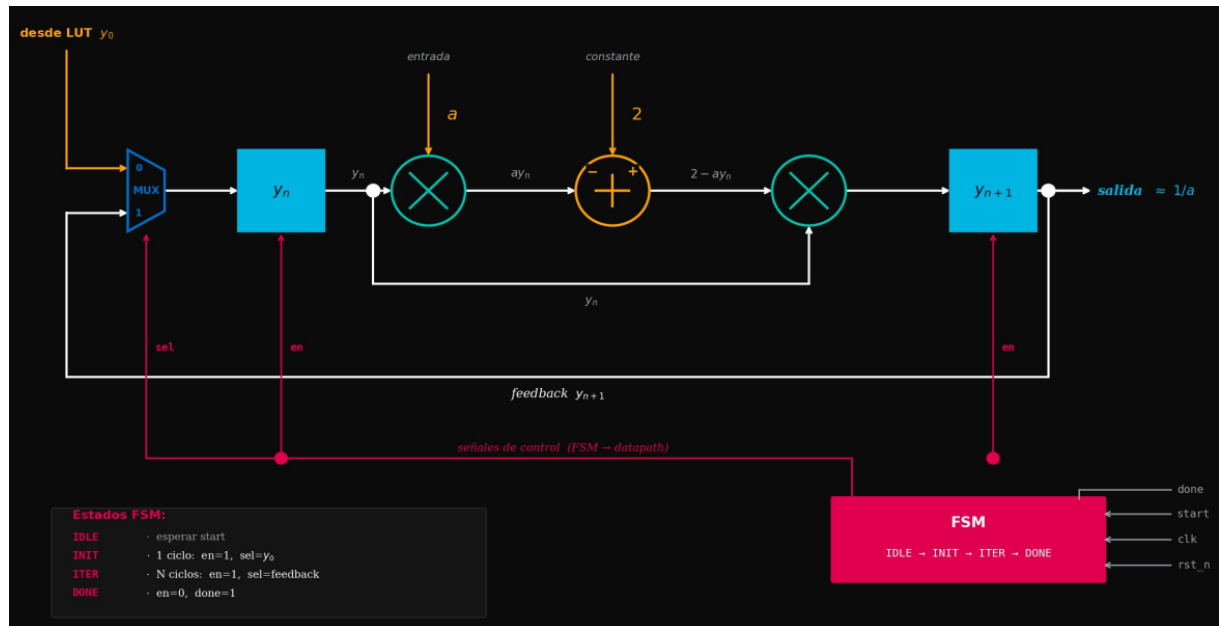


Figura 7: Esquema propuesto del divisor Newton-Raphson.

En la Fig. 8 se puede ver la salida del sistema y_o cuando la bandera **done** se levanta (se completó la operación) luego de 7 iteraciones, para $y_{inicial} = 0,1$. A simple vista se observa una precisión de 2 dígitos decimales, luego si se ve a nivel de bits, se obtiene una precisión de hasta 10 dígitos.

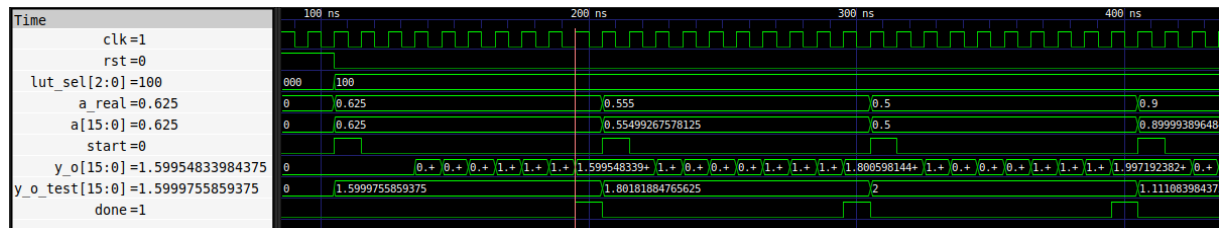


Figura 8: Formas de onda obtenidas con $a = 0,625, 0,555$ y $0,5$. $y_{inicial} = 0,1$

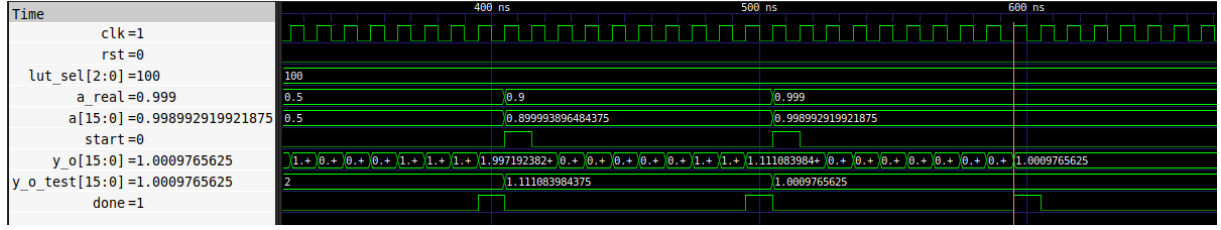


Figura 9: Formas de onda obtenidas con $a = 0,9$ y $0,999$. $y_{inicial} = 0,1$

La Tab. 2 muestra cómo evoluciona el error del sistema a medida que aumentan las iteraciones. En un escenario ideal, se esperaría que en 3 o 4 iteraciones el error ya sea imperceptible, sin embargo, la elección en este caso del valor inicial y los errores de cuantización generaron un peso importante sobre la precisión final. De todas formas, se pudo concluir que el número de bits correctos aumenta cuadráticamente con cada iteración.

Iteración	Obtenido	Error [%]	Nro. de bits correctos
1	0,1937255859	87,9	1
2	0,3639526367	77,2	1
3	0,6451416016	59,7	1
4	1,0301513672	35,6	2
5	1,3970336914	12,7	2
6	1,5742187500	1,61	6
7	1,5995483398	0,0282	10

Cuadro 2: Métricas obtenidas para $1/0,625 = 1,6$. $y_{inicial} = 0,1$

3. Conclusiones

Se demostró entonces que las arquitecturas iterativas en punto fijo permitieron implementar funciones matemáticas complejas en hardware sin recurrir a multiplicadores dedicados o memorias extensas. En las implementaciones del CORDIC, el modo rotación calculó seno y coseno en 14 iteraciones, mientras que el modo vectoring resolvió magnitud y fase simultáneamente en los cuatro cuadrantes gracias a la pre-rotación, con un error relativo inferior al 0.05 %. La comparación en la síntesis evidenció que el pipeline multiplica el área 6 veces en comparación del folded (4806 vs. 674 celdas), con la ventaja de aumentar el throughput. Por último, el divisor Newton-Raphson alcanzó 10 bits correctos tras 7 iteraciones, confirmando la convergencia cuadrática teórica.

4. Referencias

- [1] E. Monja y G. R. D. Zúñiga, *Módulo 5 - Ejercicios*, Repositorio en GitHub, Diseño Digital para Sistemas DSP, Archivos desarrollados para los ejercicios del Módulo N°5, 2026. dirección: https://github.com/ErnestMonja/Diseno_Digital_para_Sistemas_DSP/tree/main/M%C3%B3dulo%205%20-%20Ejercicios